Human Verification: `Trajectory`

Trajectory Systems and Circular Verification

Machine-generated from `lean/Trajectory.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Trajectory` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The module instantiates the general trust-system theory of `Trust.lean` to *trajectory systems*, where claims are agent events and checking steps are causal dependencies. The main result is that circular verification collapses canonical trust to $\bot$ at every event.

Contents

1 Overview **1**

2 The five-level trajectory chain **2**
2.1 Inductive type: `TLevel` . . . 2
2.2 Definition: `TLevel.le` . . . 2

3 Concrete events **3**
3.1 Inductive type: `Evt` . . . 3

4 The circular trajectory (MacBeth Day 96) **3**
4.1 Definition: `circularTraj` . . . 3
4.2 Theorem: `circularTraj_wf` . . . 4
4.3 Theorem: `circularTraj_tau_bot` . . . 4

5 The honest trajectory **4**
5.1 Definition: `honestTraj` . . . 4
5.2 Theorem: `honestTraj_wf` . . . 5
5.3 Theorems: `honest_tau_e0` through `honest_tau_e4` . . . 5

6 Translation conventions **5**

1 Overview

A *trajectory system* is a trust system (in the sense of `Containers.Trust`) whose claims are agent actions (`Evt`) and whose checking steps record causal dependencies between events. The module demonstrates two contrasting instances: a *circular* trajectory whose verification graph has a cycle (MacBeth Day 96) and an *honest* trajectory with acyclic dependencies. The canonical trust τ collapses to $\bot$ = `unsupported` on the circular instance and reaches `independent` or above on the honest instance.

2 The five-level trajectory chain

2.1 Inductive type: `TLevel`

Definition 2.1 (Trajectory trust levels)**.** The type of trajectory trust strengths is a five-element linearly ordered set:

$$\texttt{unsupported} < \texttt{shallow} < \texttt{independent} < \texttt{crossVerified} < \texttt{mechChecked}.$$

Lean.

```
inductive TLevel where
  | unsupported
  | shallow
  | independent
  | crossVerified
  | mechChecked
deriving DecidableEq, Repr
```

Haskell.

```
data TLevel
  = Unsupported
  | Shallow
  | Independent
  | CrossVerified
  | MechChecked
  deriving (Eq, Ord, Show)
```

2.2 Definition: `TLevel.le`

Definition 2.2 (Order on `TLevel`)**.** The order is defined case-by-case: `unsupported` $\le b$ for all b; `mechChecked` $\le b$ only when $b =$ `mechChecked`; and so on through the chain. The `Chain TLevel` instance packages this into a decidable linear order with $\bot =$ `unsupported`.

Lean.

```
def TLevel.le : TLevel → TLevel → Prop
  | .unsupported, _ ⇒ True
  | .shallow, .unsupported ⇒ False
  | .shallow, _ ⇒ True
  | .independent, .unsupported ⇒ False
  | .independent, .shallow ⇒ False
  | .independent, _ ⇒ True
  | .crossVerified, .mechChecked ⇒ True
  | .crossVerified, .crossVerified ⇒ True
  | .crossVerified, _ ⇒ False
  | .mechChecked, .mechChecked ⇒ True
  | .mechChecked, _ ⇒ False

instance : Chain TLevel where
  bot := .unsupported
  -- le_refl, le_trans, le_antisymm, le_total, bot_le, dec all proved
```

Haskell.

```
-- Derived Ord gives the same linear order.
-- bot = Unsupported.
```

3 Concrete events

3.1 Inductive type: `Evt`

Definition 3.1 (Events)**.** The five events e_0, e_1, e_2, e_3, e_4 are the claims of both the circular and honest trajectory systems.

Lean.
```
inductive Evt where
  | e0 | e1 | e2 | e3 | e4
deriving DecidableEq, Repr
```

Haskell.
```
data Evt = E0 | E1 | E2 | E3 | E4
  deriving (Eq, Show)
```

4 The circular trajectory (MacBeth Day 96)

4.1 Definition: `circularTraj`

Definition 4.1 (Circular trajectory system)**.** A trust system `System Evt TLevel` where each event is its own unique checking step, and the dependency graph is the chain $e_0 \leftarrow e_1 \leftarrow e_2 \leftarrow \{e_3, e_4\}$. The assigned check grades are:

$$e_0 \mapsto \texttt{unsupported}, \quad e_1, e_2 \mapsto \texttt{independent}, \quad e_3 \mapsto \texttt{shallow}, \quad e_4 \mapsto \texttt{unsupported}.$$

The name "circular" refers to the fact that the grades record verification steps that were logically circular: the supporting evidence ultimately relies on the claim being supported.

Lean.
```
def circularTraj : System Evt TLevel where
  Step := Evt
  concl := id
  prems
    | .e0 ⇒ []
    | .e1 ⇒ [.e0]
    | .e2 ⇒ [.e1]
    | .e3 ⇒ [.e2]
    | .e4 ⇒ [.e2]
  grade
    | .e0 ⇒ .unsupported
    | .e1 ⇒ .independent
    | .e2 ⇒ .independent
    | .e3 ⇒ .shallow
    | .e4 ⇒ .unsupported
  stepsInto x := [x]
  mem_stepsInto := ...
  stepsInto_mem f := .head []
```

Haskell.
```
-- A System Evt TLevel where Step = Evt,
-- prems e0=[], e1=[e0], e2=[e1], e3=[e2], e4=[e2],
-- grade e0=Unsupported, e1=Independent, e2=Independent,
--       e3=Shallow, e4=Unsupported,
-- stepsInto x = [x].
```

4.2 Theorem: `circularTraj_wf`

Theorem 4.2 (Well-foundedness of the circular premise relation)**.** The premise relation of `circularTraj` is well-founded: the dependency graph $e_0 \leftarrow e_1 \leftarrow e_2 \leftarrow \{e_3, e_4\}$ is acyclic.

Lean.
```
theorem circularTraj_wf : WellFounded circularTraj.Prem
```

Haskell.
```
-- The dependency DAG is acyclic (e0 has no premises),
-- so the premise relation is well-founded.
```

Fidelity note. Despite the system being called "circular," the *dependency graph* is acyclic (the circularity is in the *verification logic*, not the premise structure). Well-foundedness is needed to apply the certificate lemma.

4.3 Theorem: `circularTraj_tau_bot`

Theorem 4.3 (Circular verification collapses trust)**.** Canonical trust is $\bot$ at every event in the circular trajectory:

$$\forall\, e : \texttt{Evt}, \quad \tau(\texttt{circularTraj}, e) = \texttt{unsupported}.$$

The grade degradation propagates from e_0 (graded `unsupported`, no premises) through the chain: each $\tau(e_i)$ is limited by the weakest link $\tau(e_0) = \bot$.

Lean.
```
theorem circularTraj_tau_bot : ∀ (e : Evt),
    tau circularTraj circularTraj_wf e = .unsupported
```

Haskell.
```
-- prop_circularBot :: Evt → Bool
-- prop_circularBot e = tau circularTraj e == Unsupported
```

5 The honest trajectory

5.1 Definition: `honestTraj`

Definition 5.1 (Honest trajectory system)**.** The same event set and dependency structure as `circularTraj`, but with grades reflecting genuine independent verification:

$$e_0, e_1, e_2, e_4 \mapsto \texttt{independent}, \quad e_3 \mapsto \texttt{shallow}.$$

Lean.
```
def honestTraj : System Evt TLevel where
  Step := Evt
  concl := id
  prems
    | .e0 ⇒ []
    | .e1 ⇒ [.e0]
    | .e2 ⇒ [.e1]
    | .e3 ⇒ [.e2]
    | .e4 ⇒ [.e2]
  grade
    | .e0 ⇒ .independent
    | .e1 ⇒ .independent
    | .e2 ⇒ .independent
    | .e3 ⇒ .shallow
    | .e4 ⇒ .independent
  stepsInto x := [x]
```

```
  mem_stepsInto := ...
  stepsInto_mem f := .head []
```

Haskell.

```
-- Same structure as circularTraj but grade e0 = Independent.
```

5.2 Theorem: `honestTraj_wf`

Theorem 5.2 (Well-foundedness of the honest premise relation)**.** The premise relation of `honestTraj` is well-founded.

Lean.

```
theorem honestTraj_wf : WellFounded honestTraj.Prem
```

Haskell.

```
-- Same DAG; acyclic and hence well-founded.
```

5.3 Theorems: `honest_tau_e0` through `honest_tau_e4`

Theorem 5.3 (Canonical trust in the honest trajectory)**.** The canonical trust values in the honest trajectory are:

$$\begin{aligned}\tau(e_0) &= \texttt{independent},\\ \tau(e_1) &= \texttt{independent},\\ \tau(e_2) &= \texttt{independent},\\ \tau(e_3) &= \texttt{shallow},\\ \tau(e_4) &= \texttt{independent}.\end{aligned}$$

At each event, τ is the meet of the grade along its unique dependency chain from e_0. e_3 receives only `shallow` because its step is graded `shallow`; all other events reach `independent`.

Lean.

```
theorem honest_tau_e0 :
    tau honestTraj honestTraj_wf .e0 = .independent

theorem honest_tau_e1 :
    tau honestTraj honestTraj_wf .e1 = .independent

theorem honest_tau_e2 :
    tau honestTraj honestTraj_wf .e2 = .independent

theorem honest_tau_e3 :
    tau honestTraj honestTraj_wf .e3 = .shallow

theorem honest_tau_e4 :
    tau honestTraj honestTraj_wf .e4 = .independent
```

Haskell.

```
-- prop_honestTau :: Evt → TLevel → Bool
-- The expected values match the weakest-link computation through the
   DAG.
```

6 Translation conventions

1. **Proof bodies omitted.** Lean machine-checks all proofs. Human verification covers only the *statements*.

2. **Chain instance.** The `instance : Chain TLevel` bundles reflexivity, transitivity, antisymmetry, totality, bot-below-all, and decidability. In Haskell this corresponds to `Eq` + `Ord` with the natural derived instance.

3. **Well-foundedness.** The certificate lemma (`sound_le_tau` in `Trust.lean`) requires an explicit `WellFounded` proof. The circular trajectory's DAG is acyclic (the circularity is in the verification logic, not the graph), so `WellFounded` holds; the Lean proof constructs accessibility witnesses node by node.

4. **Private helper theorems.** The leaf-to-root computation of τ for each event (`tau_circ_e0`, `tau_circ_e1`, etc.) are `private` helpers leading to the exported `circularTraj_tau_bot`. They are stated but not expanded here.